

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM



MÔN LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG
BÀI TẬP THỰC HÀNH 4

GVHD: Nguyễn Ngọc Quý

Sinh viên thực hiện: Trịnh Minh Hiếu

MSSV: 25520560

This image shows a full page of white paper with horizontal dotted lines, typical of primary school handwriting practice paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings present.

(Ký tên và ghi rõ họ tên)

MỤC LỤC

1.1	Số phức	6
-----	---------------	---

DANH MỤC BẢNG

No table of figures entries found.

DANH MỤC HÌNH ẢNH

No table of figures entries found.

NỘI DUNG BÀI LÀM

1.1 Số phức (Complex)

- Thuộc tính: re, im
- Phương thức: phương thức khởi tạo, operator+(), operator-(), operator*(), operator/(), operator==(), operator!=(), operator<<(), operator>>()
- Class diagram:

Complex
-re, im: double
<pre> +Complex(r: double = 0, i: double = 0) +operator+(other: Complex) : Complex +operator-(other: Complex) : Complex +operator*(other: Complex) : Complex +operator/(other: Complex) : Complex +operator==(other: Complex) : bool +operator!=(other: Complex) : bool <> operator<<(os: ostream, z: Complex) : ostream <> operator>>(is: istream, z: Complex) : istream +~Complex()</pre>

- Code:
- **Complex.h:**

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

```
#ifndef COMPLEX_H
#define COMPLEX_H
#include <iostream>
class Complex
{
private:
    double re;
    double im;
public:
    Complex(double r = 0, double i = 0);
    Complex operator+ (const Complex& other) const;
    Complex operator- (const Complex& other) const;
    Complex operator* (const Complex& other) const;
    Complex operator/ (const Complex& other) const;
    bool operator== (const Complex& other) const;
    bool operator!= (const Complex& other) const;
    friend std::ostream& operator<<(std::ostream& os, const Complex& z);
    friend std::istream& operator>>(std::istream& is, Complex& z);
    ~Complex();
};

#endif // COMPLEX_H
```

- **Complex.cpp:**

```
Complex.cpp
#include "Complex.h"
#include <limits>
#include <iostream>
Complex::Complex(double r, double i): re(r), im(i) {}
Complex Complex::operator+ (const Complex& other) const
{
    return Complex (re + other.re, im + other.im);
}
Complex Complex::operator- (const Complex& other) const
{
    return Complex (re - other.re, im - other.im);
}
Complex Complex::operator* (const Complex& other) const
{
    return Complex (re * other.re - im * other.im, re * other.im + im * other.re);
}
Complex Complex::operator/ (const Complex& other) const
{
    double denom = other.re * other.re + other.im * other.im;
    if (denom == 0) return Complex(std::numeric_limits<double>::quiet_NaN(),
                                   std::numeric_limits<double>::quiet_NaN());
    return Complex ((re * other.re + im * other.im)/denom, (im * other.re - re * other.im)/denom);
}
bool Complex::operator== (const Complex& other) const
{
    return (re == other.re && im == other.im);
}
bool Complex::operator!= (const Complex& other) const
{
    return !(*this == other);
}
std::ostream& operator<<(std::ostream& os, const Complex& z)
{
    os << z.re << (z.im >= 0 ? " + " : " - ") << (z.im >= 0 ? z.im : -z.im) << "i";
    return os;
}
std::istream& operator>>(std::istream& is, Complex& z)
{
    is >> z.re >> z.im;
    return is;
}
Complex::~~Complex()
{
    //dtor
}
```

- **main.cpp:**

```

#include <iostream>
#include "Complex.h"
using namespace std;

int main()
{
    Complex z1, z2;
    cin >> z1;
    cin >> z2;
    cout << "Tong: " << z1 + z2 << endl;
    cout << "Hieu: " << z1 - z2 << endl;
    cout << "Tich: " << z1 * z2 << endl;
    cout << "Thuong: " << z1 / z2 << endl;
    cout << "Co bang nhau khong? (1 la co, 0 la khong): " << (z1 == z2) << endl;
    cout << "Co khac nhau khong? (1 la co, 0 la khong): " << (z1 != z2) << endl;
    return 0;
}

```

- Test cases:

Test case	Input	Output
1	Số phức 1: phần thực 3, phần ảo 4 Số phức 2: phần thực 5, phần ảo 6	<pre> 3 4 5 6 Tong: 8 + 10i Hieu: -2 - 2i Tich: -9 + 38i Thuong: 0.639344 + 0.0327869i Co bang nhau khong? (1 la co, 0 la khong): 0 Co khac nhau khong? (1 la co, 0 la khong): 1 Process returned 0 (0x0) execution time : 2.153 s Press any key to continue. </pre>
2	Số phức 1: phần thực 4, phần ảo 2 Số phức 2: phần thực 2, phần ảo 1	<pre> 4 2 2 1 Tong: 6 + 3i Hieu: 2 + 1i Tich: 6 + 8i Thuong: 2 + 0i Co bang nhau khong? (1 la co, 0 la khong): 0 Co khac nhau khong? (1 la co, 0 la khong): 1 Process returned 0 (0x0) execution time : 2.704 s Press any key to continue. </pre>

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

3	Số phức 1: phần thực 0, phần ảo 0 Số phức 2: phần thực 0, phần ảo 0	<pre> 0 0 0 0 Tong: 0 + 0i Hieu: 0 + 0i Tich: 0 + 0i Thuong: nan - nani Co bang nhau khong? (1 la co, 0 la khong): 1 Co khac nhau khong? (1 la co, 0 la khong): 0 Process returned 0 (0x0) execution time : 2.279 s Press any key to continue. </pre>
4	Số phức 1: phần thực 0, phần ảo 0 Số phức 2: phần thực 3, phần ảo 6	<pre> 0 0 3 6 Tong: 3 + 6i Hieu: -3 - 6i Tich: 0 + 0i Thuong: 0 + 0i Co bang nhau khong? (1 la co, 0 la khong): 0 Co khac nhau khong? (1 la co, 0 la khong): 1 Process returned 0 (0x0) execution time : 3.660 s Press any key to continue. </pre>
5	Số phức 1: phần thực 2, phần ảo 3 Số phức 2: phần thực 5, phần ảo 7	<pre> 2 3 5 7 Tong: 7 + 10i Hieu: -3 - 4i Tich: -11 + 29i Thuong: 0.418919 + 0.0135135i Co bang nhau khong? (1 la co, 0 la khong): 0 Co khac nhau khong? (1 la co, 0 la khong): 1 Process returned 0 (0x0) execution time : 3.065 s Press any key to continue. </pre>

1.2 Phân số (Fractions)

- Thuộc tính: num, dem
- Phương thức: phương thức khởi tạo, operator+(), operator-(), operator*(), operator/(), operator==(), operator!=(), operator<<(), operator>>()
- Class diagram:

Fractions

-num : int

-dem : int

+Fractions(num : int, dem : int)

+operator+(other : Fractions) : Fractions

+operator-(other : Fractions) : Fractions

+operator*(other : Fractions) : Fractions

+operator/(other : Fractions) : Fractions

+operator==(other : Fractions) : bool

+operator!=(other : Fractions) : bool

+~Fractions()

<> operator<<(os : ostream, f : Fractions) : ostream

<> operator>>(is : istream, f : Fractions) : istream

- Code:

- **Fractions.h:**

```
#ifndef FRACTIONS_H
#define FRACTIONS_H
#include <iostream>
using namespace std;
class Fractions
{
private:
    int num;
    int dem;
public:
    Fractions(int num = 0, int dem = 1);
    Fractions operator+ (const Fractions& other) const;
    Fractions operator- (const Fractions& other) const;
    Fractions operator* (const Fractions& other) const;
    Fractions operator/ (const Fractions& other) const;
    bool operator== (const Fractions& other) const;
    bool operator!= (const Fractions& other) const;
    friend std::ostream& operator<<(std::ostream& os, const Fractions& f);
    friend std::istream& operator>>(std::istream& is, Fractions& f);
    ~Fractions();
};

#endif // FRACTIONS_H
```

- **Fractions.cpp:**

```
#include "Fractions.h"
#include <limits>
#include <iostream>
#include <numeric>
#include <stdexcept>
Fractions::Fractions(int n, int d): num(n), dem(d) {}
Fractions Fractions::operator+ (const Fractions& other) const
{
    Fractions sum;
    sum.num = num * other.dem + dem * other.num;
    sum.dem = dem * other.dem;
    return Fractions (sum.num/gcd(sum.num, sum.dem), sum.dem/gcd(sum.num, sum.dem));
}
Fractions Fractions::operator- (const Fractions& other) const
{
    Fractions diff;
    diff.num = num * other.dem - dem * other.num;
    diff.dem = dem * other.dem;
    return Fractions (diff.num/gcd(diff.num, diff.dem), diff.dem/gcd(diff.num, diff.dem));
}
Fractions Fractions::operator* (const Fractions& other) const
{
    Fractions prod;
    prod.num = num * other.num;
    prod.dem = dem * other.dem;
    return Fractions (prod.num/gcd(prod.num, prod.dem), prod.dem/gcd(prod.num, prod.dem));
}
Fractions Fractions::operator/ (const Fractions& other) const
{
    if (dem == 0 || other.dem == 0 || other.num == 0) return Fractions (std::numeric_limits<int>::quiet_NaN(), std::numeric_limits<int>::quiet_NaN());
    else
    {
        Fractions quo;
        quo.num = num * other.dem;
        quo.dem = dem * other.num;
        return Fractions (quo.num/gcd(quo.num, quo.dem), quo.dem/gcd(quo.num, quo.dem));
    }
}
bool Fractions::operator== (const Fractions& other) const
{
    return (num == other.num && dem == other.dem);
}
bool Fractions::operator!= (const Fractions& other) const
{
    return !(*this == other);
}

bool Fractions::operator!= (const Fractions& other) const
{
    return !(*this == other);
}

std::ostream& operator<<(std::ostream& os, const Fractions& f)
{
    if (f.dem < 0) os << -f.num << "/" << -f.dem;
    else if (f.dem == 1) cout << f.num;
    else os << f.num << "/" << f.dem;
    return os;
}

std::istream& operator>>(std::istream& is, Fractions& f)
{
    is >> f.num >> f.dem;
    return is;
}

Fractions::~Fractions()
{
    //dtor
}
```

- **main.cpp:**

```

#include <iostream>
#include "Fractions.h"
using namespace std;

int main()
{
    Fractions f1, f2;
    cin >> f1;
    cin >> f2;
    cout << "Tong: " << f1 + f2 << endl;
    cout << "Hieu: " << f1 - f2 << endl;
    cout << "Tich: " << f1 * f2 << endl;
    cout << "Thuong: " << f1 / f2 << endl;
    cout << "Co bang nhau khong? (1 la co, 0 la khong): " << (f1 == f2) << endl;
    cout << "Co khac nhau khong? (1 la co, 0 la khong): " << (f1 != f2) << endl;
    return 0;
}

```

- Test cases:

Test case	Input	Output
1	Phân số 1: tử 3, mẫu 4 Phân số 2: tử 5, mẫu 6	<pre> 3 4 5 6 Tong: 19/12 Hieu: -1/12 Tich: 5/8 Thuong: 9/10 Co bang nhau khong? (1 la co, 0 la khong): 0 Co khac nhau khong? (1 la co, 0 la khong): 1 Process returned 0 (0x0) execution time : 2.748 s Press any key to continue. </pre>
2	Phân số 1: tử 4, mẫu 2 Phân số 2: tử 2, mẫu 1	<pre> 4 2 2 1 Tong: 4 Hieu: 0 Tich: 4 Thuong: 1 Co bang nhau khong? (1 la co, 0 la khong): 0 Co khac nhau khong? (1 la co, 0 la khong): 1 Process returned 0 (0x0) execution time : 1.778 s Press any key to continue. </pre>
3	Phân số 1: tử 0, mẫu 0 Phân số 2: tử 0, mẫu 0	<pre> 0 0 0 0 Tong: Process returned -1073741676 (0xC0000094) execution time : 2.361 s Press any key to continue. </pre>

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

4	Phân số 1: tử 0, mẫu 0 Phân số 2: tử 3, mẫu 6	<pre>0 0 3 6 Tong: Process returned -1073741676 (0xC0000094) execution time : 2.914 s Press any key to continue.</pre>
5	Phân số 1: tử 2, mẫu 3 Phân số 2: tử 5, mẫu 7	<pre>2 3 5 7 Tong: 29/21 Hieu: -1/21 Tich: 10/21 Thuong: 14/15 Co bang nhau khong? (1 la co, 0 la khong): 0 Co khac nhau khong? (1 la co, 0 la khong): 1 Process returned 0 (0x0) execution time : 1.646 s Press any key to continue.</pre>

1.3 CTime

- Thuộc tính: gio, phut, giay
- Phương thức: chuanHoa, phương thức khởi tạo, operator+(), operator-(), operator++(), operator--(), operator++(int), operator--(int), operator<<, operator>>
- Class diagram:

CTime

-gio: int
-phut: int
-giay: int

-chuanHoa() : void
+CTime(h: int, m: int, s: int)
+operator+(s: int) : CTime
+operator-(s: int) : CTime
+operator++() : CTime&
+operator--() : CTime&
+operator++(int) : CTime
+operator--(int) : CTime
<> operator<<(os: ostream, t: CTime) : ostream
<> operator>>(is: istream, t: CTime) : istream

- Code:
- **CTime.h:**

```

#ifndef CTIME_H
#define CTIME_H
#include <iostream>
class CTime {
private:
    int gio, phut, giay;
    void chuanHoa();
public:
    CTime(int h = 0, int m = 0, int s = 0);
    CTime operator+(int s) const;
    CTime operator-(int s) const;
    CTime& operator++();
    CTime& operator--();
    CTime operator++(int);
    CTime operator--(int);
    friend std::ostream& operator<<(std::ostream& os, const CTime& t);
    friend std::istream& operator>>(std::istream& is, CTime& t);
};
#endif

```

- CTime.cpp:

```

#include "CTime.h"
CTime::CTime(int h, int m, int s) : gio(h), phut(m), giay(s) { chuanHoa(); }
void CTime::chuanHoa() {
    long tong = (long)gio * 3600 + phut * 60 + giay;
    if (tong < 0) tong = 0;
    gio = (tong / 3600) % 24;
    phut = (tong % 3600) / 60;
    giay = tong % 60;
}
CTime CTime::operator+(int s) const { return CTime(gio, phut, giay + s); }
CTime CTime::operator-(int s) const { return CTime(gio, phut, giay - s); }
CTime& CTime::operator++() { giay++; chuanHoa(); return *this; }
CTime CTime::operator++(int) { CTime tmp = *this; ++(*this); return tmp; }
CTime& CTime::operator--() { giay--; chuanHoa(); return *this; }
CTime CTime::operator--(int) { CTime tmp = *this; --(*this); return tmp; }
std::ostream& operator<<(std::ostream& os, const CTime& t) {
    if (t.gio < 10) os << "0"; os << t.gio << ":";
    if (t.phut < 10) os << "0"; os << t.phut << ":";
    if (t.giay < 10) os << "0"; os << t.giay;
    return os;
}
std::istream& operator>>(std::istream& is, CTime& t) {
    is >> t.gio >> t.phut >> t.giay;
    t.chuanHoa();
    return is;
}

```

- main.cpp:

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

```
#include "CTime.h"
#include <iostream>
#include <chrono>
#include <thread>
#include <windows.h>

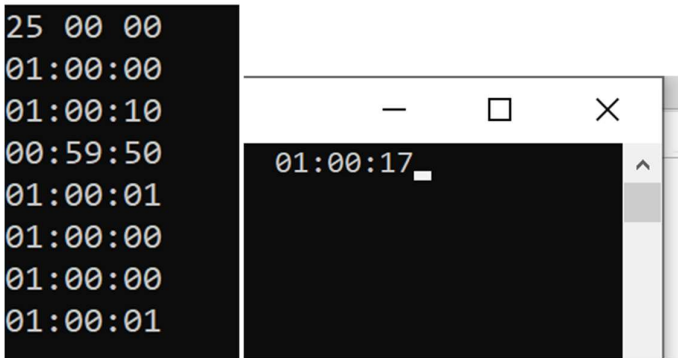
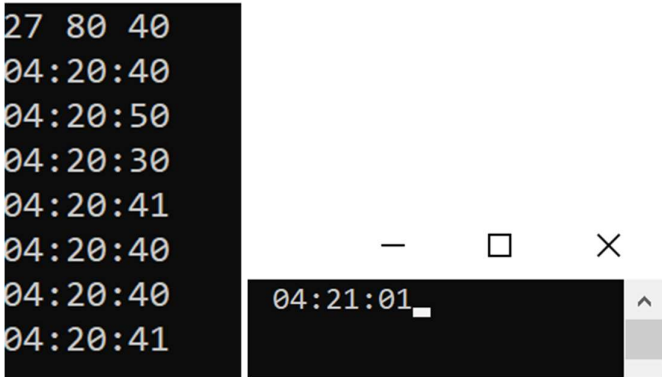
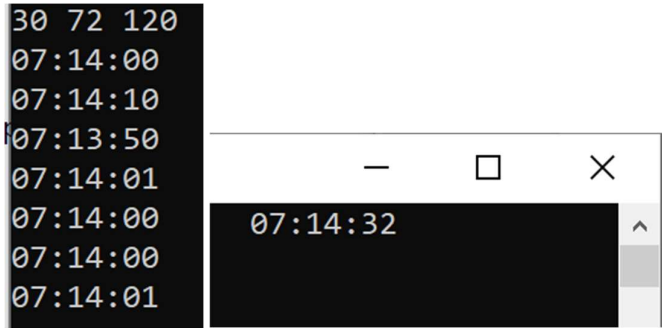
void pos(int x, int y) {
    COORD c = {(SHORT)x, (SHORT)y};
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), c);
}

int main() {
    CTime t;
    std::cin >> t;
    std::cout << t << std::endl;
    std::cout << (t + 10) << std::endl;
    std::cout << (t - 10) << std::endl;
    std::cout << ++t << std::endl;
    std::cout << --t << std::endl;
    std::cout << t++ << std::endl;
    std::cout << t-- << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(2));
    system("cls");
    while (true) {
        pos(100, 0);
        std::cout << t << std::flush;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        ++t;
    }
    return 0;
}
```

- Test cases:

Test case	Input	Output
1	20:20:20	 The output shows a list of timestamps: 20 20 20, 20:20:20, 20:20:30, 20:20:10, 20:20:21, 20:20:20, 20:20:20, 20:20:21. To the right is a screenshot of a console window titled '20:20:21' with a cursor at the end of the line.

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

2	25:00:00	
3	27:80:40	
4	30:72:120	

1.4 CDate

- Thuộc tính: d, m, y
- Phương thức: laNamNhuan, ngayTrongThang, toDays, fromDays, phương thức khởi tạo, operator+(), operator-(days), operator++(), operator--(), operator++(int), operator--(int), operator<<, operator>>
- Class diagram:

CDate**-d: int****-m: int****-y: int****-laNamNhuan(y: int) : : bool****-ngayTrongThang(m: int, y: int) : : int****-toDays() : : long****-fromDays(days: long) : : CDate****+CDate(d: int, m: int, y: int)****+operator+(days: int) : : CDate****+operator-(days: int) : : CDate****+operator-(other: CDate) : : long****+operator++() : : CDate&****+operator++(int) : : CDate****+operator--() : : CDate&****+operator--(int) : : CDate****<> operator<<(os: ostream, dt: CDate) : : ostream****<> operator>>(is: istream, dt: CDate) : : istream****- Code:****• CDate.h:**

```

#ifndef CDATE_H
#define CDATE_H
#include <iostream>
class CDate {
private:
    int d, m, y;
    bool laNamNhuan(int y) const;
    int ngayTrongThang(int m, int y) const;
    long toDays() const;
    static CDate fromDays(long days);
public:
    CDate(int d = 1, int m = 1, int y = 1);
    CDate operator+(int days) const;
    CDate operator-(int days) const;
    long operator-(const CDate& other) const;
    CDate& operator++();
    CDate operator++(int);
    CDate& operator--();
    CDate operator--(int);
    friend std::ostream& operator<<(std::ostream& os, const CDate& dt);
    friend std::istream& operator>>(std::istream& is, CDate& dt);
};
#endif

```

- CDate.cpp:

```

#include "CDate.h"
CDate::CDate(int d, int m, int y) : d(d), m(m), y(y) {}
bool CDate::laNamNhuan(int y) const { return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0); }
int CDate::ngayTrongThang(int m, int y) const {
    int ds[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    return (m == 2 && laNamNhuan(y)) ? 29 : ds[m];
}
long CDate::toDays() const {
    long total = d;
    for (int i = 1; i < y; ++i) total += (laNamNhuan(i) ? 366 : 365);
    for (int i = 1; i < m; ++i) total += ngayTrongThang(i, y);
    return total;
}
CDate CDate::fromDays(long days) {
    int y = 1, m = 1;
    while (days > (CDate(1, 1, y).laNamNhuan(y) ? 366 : 365)) {
        days -= (CDate(1, 1, y).laNamNhuan(y) ? 366 : 365);
        y++;
    }
    CDate tmp(1, 1, y);
    while (days > tmp.ngayTrongThang(m, y)) {
        days -= tmp.ngayTrongThang(m, y);
        m++;
    }
    return CDate((int)days, m, y);
}
CDate CDate::operator+(int days) const { return fromDays(toDays() + days); }
CDate CDate::operator-(int days) const { return fromDays(toDays() - days); }
long CDate::operator-(const CDate& other) const { return toDays() - other.toDays(); }
CDate& CDate::operator++() { *this = *this + 1; return *this; }
CDate CDate::operator++(int) { CDate tmp = *this; ++(*this); return tmp; }
CDate& CDate::operator--() { *this = *this - 1; return *this; }
CDate CDate::operator--(int) { CDate tmp = *this; --(*this); return tmp; }
std::ostream& operator<<(std::ostream& os, const CDate& dt) {
    os << (dt.d < 10 ? "0" : "") << dt.d << "/" << (dt.m < 10 ? "0" : "") << dt.m << "/" << dt.y;
    return os;
}
std::istream& operator>>(std::istream& is, CDate& dt) {
    int d, m, y;
    if (is >> d >> m >> y) {
        dt = CDate(d, m, y);
    }
    return is;
}

```

- main.cpp:

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

```
#include "CDate.h"
#include <iostream>
#include <cmath>
#include <iomanip>

int main() {
    CDate gui, rut;
    double goc, laiNam;

    if (!(std::cin >> gui >> rut >> goc >> laiNam)) return 1;

    long n = rut - gui;
    if (n < 0) return 1;

    double r = (laiNam / 100.0) / 365.0;
    double laiDon = goc * r * n;
    double tongDon = goc + laiDon;
    double tongKep = goc * std::pow(1 + r, n);
    double laiKep = tongKep - goc;

    std::cout << "So ngay: " << n << std::endl;

    std::cout << "--- Lai Don ---" << std::endl;
    std::cout << "Tien lai: " << (long long)laiDon << std::endl;
    std::cout << "Tong: " << (long long)tongDon << std::endl;

    std::cout << "--- Lai Kep ---" << std::endl;
    std::cout << "Tien lai: " << (long long)laiKep << std::endl;
    std::cout << "Tong: " << (long long)tongKep << std::endl;

    return 0;
}
```

- Test cases:

Test case	Input	Output
1	Ngày gửi: 24/1/2025 Ngày rút: 24/1/2029 Góc: 1000 Lãi: 5%/năm	<pre>24 1 2025 24 1 2029 1000 5 So ngay: 1461 --- Lai Don --- Tien lai: 200 Tong: 1200 --- Lai Kep --- Tien lai: 221 Tong: 1221</pre>

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

2	Ngày gửi: 20/1/2025 Ngày rút: 36/1/2028 Gốc: 2000 Lãi: 7.5%/năm	<pre> 20 1 2025 36 1 2028 2000 7.5 So ngay: 1111 --- Lai Don --- Tien lai: 456 Tong: 2456 --- Lai Kep --- Tien lai: 512 Tong: 2512 Process returned 0 (0x0) execution time : 8.837 s Press any key to continue. </pre>
3	Ngày gửi: 0/24/2025 Ngày rút: 0/24/2027 Gốc: 1500 Lãi: 6%/năm	<pre> 0 24 2025 0 24 2027 1500 6 So ngay: 718 --- Lai Don --- Tien lai: 177 Tong: 1677 --- Lai Kep --- Tien lai: 187 Tong: 1687 Process returned 0 (0x0) execution time : 10.082 s Press any key to continue. </pre>
4	Ngày gửi: -2/-3/2025 Ngày rút: -2/-3/2027 Gốc: 1500 Lãi: 6%/năm	<pre> -2 -3 2025 -2 -3 2026 1000 8 So ngay: 365 --- Lai Don --- Tien lai: 80 Tong: 1080 --- Lai Kep --- Tien lai: 83 Tong: 1083 Process returned 0 (0x0) execution time : 10.619 s Press any key to continue. </pre>

1.5 Đa thức (DaThuc)

- Thuộc tính: heSo
- Phương thức: rutGon, phương thức khởi tạo, operator+(), operator-(), operator*(), operator/(), operator%(), tinh(), operator>>, operator<<
- Class diagram:

DaThuc

-heSo: vector<double>

-rutGon() : void

+DaThuc(bac: int)

+operator+(other: DaThuc) : DaThuc

+operator-(other: DaThuc) : DaThuc

+operator*(other: DaThuc) : DaThuc

+operator/(other: DaThuc) : DaThuc

+operator%(other: DaThuc) : DaThuc

+tinh(x: double) : double

<> operator<<(os: ostream, dt: DaThuc) : ostream

<> operator>>(is: istream, dt: DaThuc) : istream

- Code:

- **DaThuc.h:**

```
#ifndef DATHUC_H
#define DATHUC_H
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>

class DaThuc {
private:
    std::vector<double> heSo;
    void rutGon();
public:
    DaThuc(int bac = 0);
    DaThuc operator+(const DaThuc& other) const;
    DaThuc operator-(const DaThuc& other) const;
    DaThuc operator*(const DaThuc& other) const;
    DaThuc operator/(const DaThuc& other) const;
    DaThuc operator%(const DaThuc& other) const;
    double tinh(double x) const;
    friend std::ostream& operator<<(std::ostream& os, const DaThuc& dt);
    friend std::istream& operator>>(std::istream& is, DaThuc& dt);
};
#endif
```

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

- **DaThuc.cpp:**

```

#include "DaThuc.h"
using namespace std;

DaThuc::DaThuc(int bac) {
    heSo.resize(bac < 0 ? 1 : bac + 1, 0);
}

void DaThuc::rutGon() {
    while (heSo.size() > 1 && heSo.back() == 0) heSo.pop_back();
}

DaThuc DaThuc::operator+(const DaThuc& other) const {
    DaThuc res(max(heSo.size(), other.heSo.size()) - 1);
    for (size_t i = 0; i < res.heSo.size(); ++i) {
        if (i < heSo.size()) res.heSo[i] += heSo[i];
        if (i < other.heSo.size()) res.heSo[i] += other.heSo[i];
    }
    res.rutGon();
    return res;
}

DaThuc DaThuc::operator-(const DaThuc& other) const {
    DaThuc res(max(heSo.size(), other.heSo.size()) - 1);
    for (size_t i = 0; i < res.heSo.size(); ++i) {
        if (i < heSo.size()) res.heSo[i] += heSo[i];
        if (i < other.heSo.size()) res.heSo[i] -= other.heSo[i];
    }
    res.rutGon();
    return res;
}

DaThuc DaThuc::operator*(const DaThuc& other) const {
    DaThuc res(heSo.size() + other.heSo.size() - 2);
    for (size_t i = 0; i < heSo.size(); ++i)
        for (size_t j = 0; j < other.heSo.size(); ++j)
            res.heSo[i + j] += heSo[i] * other.heSo[j];
    res.rutGon();
    return res;
}

DaThuc DaThuc::operator/(const DaThuc& other) const {
    if (other.heSo.empty() || other.heSo.back() == 0) return DaThuc(0);
    DaThuc r = *this, q(max(0, (int)(heSo.size() - other.heSo.size())));
    if (heSo.size() < other.heSo.size()) return DaThuc(0);
    for (int i = r.heSo.size() - other.heSo.size(); i >= 0; --i) {
        double f = r.heSo[i + other.heSo.size() - 1] / other.heSo.back();
    }
}

```



```

    if (heSo.size() < other.heSo.size()) return DaThuc(0);
    for (int i = r.heSo.size() - other.heSo.size(); i >= 0; --i) {
        double f = r.heSo[i + other.heSo.size() - 1] / other.heSo.back();
        q.heSo[i] = f;
        for (size_t j = 0; j < other.heSo.size(); ++j)
            r.heSo[i + j] -= f * other.heSo[j];
    }
    q.rutGon();
    return q;
}

DaThuc DaThuc::operator%(const DaThuc& other) const {
    if (heSo.size() < other.heSo.size()) return *this;
    DaThuc res = *this - ((*this / other) * other);
    res.rutGon();
    return res;
}

double DaThuc::tinh(double x) const {
    double res = 0;
    for (int i = heSo.size() - 1; i >= 0; --i) res = res * x + heSo[i];
    return res;
}

ostream& operator<<(ostream& os, const DaThuc& dt) {
    bool f = true;
    for (int i = dt.heSo.size() - 1; i >= 0; --i) {
        if (dt.heSo[i] == 0) continue;
        if (!f && dt.heSo[i] > 0) os << " + ";
        if (dt.heSo[i] < 0) os << " - ";
        if (abs(dt.heSo[i]) != 1 || i == 0) os << abs(dt.heSo[i]);
        if (i > 0) os << "x";
        if (i > 1) os << "^" << i;
        f = false;
    }
    return f ? os << "0" : os;
}

istream& operator>>(istream& is, DaThuc& dt) {
    int b; is >> b;
    dt.heSo.assign(b + 1, 0);
    for (int i = b; i >= 0; --i) is >> dt.heSo[i];
    dt.rutGon();
    return is;
}

```

- **main.cpp:**

```

#include "DaThuc.h"
using namespace std;

int main() {
    DaThuc p1, p2;
    double x;
    cin >> p1 >> p2 >> x;
    cout << "Tong: " << p1 + p2 << endl;
    cout << "Hieu: " << p1 - p2 << endl;
    cout << "Tich: " << p1 * p2 << endl;
    cout << "Thuong: " << p1 / p2 << endl;
    cout << "Du: " << p1 % p2 << endl;
    cout << "Gia tri cua p1 tai x = " << x << " la: " << p1.tinh(x) << endl;
    cout << "Gia tri cua p2 tai x = " << x << " la: " << p2.tinh(x) << endl;
    return 0;
}

```

- **Test cases:**

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Test case	Input	Output
1	Đa thức 1: $x^3 + 3x^2 + 3x + 1$ Đa thức 2: $x^2 + 2x + 1$ Cần tính giá trị tại $x = 4$	<pre> 3 1 3 3 1 2 1 2 1 4 Tong: x^3 + 4x^2 + 5x + 2 Hieu: x^3 + 2x^2 + x Tich: x^5 + 5x^4 + 10x^3 + 10x^2 + 5x + 1 Thuong: x + 1 Du: 0 Gia tri cua p1 tai x = 4 la: 125 Gia tri cua p2 tai x = 4 la: 25 </pre>
2	Đa thức 1: $x^4 + 4x^3 + 6x^2 + 4x + 2$ Đa thức 2: $x^2 + 2x$ Cần tính giá trị tại $x = 3$	<pre> 4 1 4 6 4 2 2 1 2 0 3 Tong: x^4 + 4x^3 + 7x^2 + 6x + 2 Hieu: x^4 + 4x^3 + 5x^2 + 2x + 2 Tich: x^6 + 6x^5 + 14x^4 + 16x^3 + 10x^2 + 4x Thuong: x^2 + 2x + 2 Du: 2 Gia tri cua p1 tai x = 3 la: 257 Gia tri cua p2 tai x = 3 la: 15 Process returned 0 (0x0) execution time : 13.158 s Press any key to continue. </pre>
3	Đa thức 1: $x^3 + 3x + 1$ Đa thức 2: $x + 1$ Cần tính giá trị tại $x = 2$	<pre> 3 1 0 3 1 1 1 1 p2 Tong: x^3 + 4x + 2 Hieu: x^3 + 2x Tich: x^4 + x^3 + 3x^2 + 4x + 1 Thuong: x^2 - x + 4 Du: - 3 Gia tri cua p1 tai x = 2 la: 15 Gia tri cua p2 tai x = 2 la: 3 Process returned 0 (0x0) execution time : 14.715 s Press any key to continue. </pre>

4	Đa thức 1: $x^4 + 2025x^2 + 2024x + 2025$ Đa thức 2: $x^2 + x + 1$ Cần tính giá trị tại $x = 1.5$	<pre> 4 1 0 2025 2024 2025 2 1 1 1 1.5 Tong: x^4 + 2026x^2 + 2025x + 2026 Hieu: x^4 + 2024x^2 + 2023x + 2024 Tich: x^6 + x^5 + 2026x^4 + 4049x^3 + 6074x^2 + 4049x + 2025 Thuong: x^2 - x + 2025 Du: 0 Gia tri cua p1 tai x = 1.5 la: 9622.31 Gia tri cua p2 tai x = 1.5 la: 4.75 Process returned 0 (0x0) execution time : 9.897 s Press any key to continue. </pre>
---	---	--

1.6 CVector

- Thuộc tính: v
- Phương thức: phương thức khởi tạo, operator+(), operator-(), operator*(CVector), operator*(double), module(), operator<<, operator>>
- Class diagram:

CVector
-v: vector<double>
+CVector(n: int) +operator+(other: CVector) : : CVector +operator-(other: CVector) : : CVector +operator*(other: CVector) : : double +operator*(scalar: double) : : CVector +module() : : double <> operator<<(os: ostream, cv: CVector) : : ostream <> operator>>(is: istream, cv: CVector) : : istream

- Code:
- CVector.h:

```

#ifndef CVECTOR_H
#define CVECTOR_H
#include <iostream>
#include <vector>
#include <cmath>

class CVector {
    friend class CMatrix;
private:
    std::vector<double> v;
public:
    CVector(int n = 0);
    CVector operator+(const CVector& other) const;
    CVector operator-(const CVector& other) const;
    double operator*(const CVector& other) const;
    CVector operator*(double scalar) const;
    double module() const;
    friend std::ostream& operator<<(std::ostream& os, const CVector& cv);
    friend std::istream& operator>>(std::istream& is, CVector& cv);
};
#endif

```

- CVector.cpp:

```

#include "CVector.h"
using namespace std;

CVector::CVector(int n) { v.assign(n, 0); }

CVector CVector::operator+(const CVector& other) const {
    if (v.size() != other.v.size()) return CVector(0);
    CVector res(v.size());
    for (size_t i = 0; i < v.size(); ++i) res.v[i] = v[i] + other.v[i];
    return res;
}

CVector CVector::operator-(const CVector& other) const {
    if (v.size() != other.v.size()) return CVector(0);
    CVector res(v.size());
    for (size_t i = 0; i < v.size(); ++i) res.v[i] = v[i] - other.v[i];
    return res;
}

double CVector::operator*(const CVector& other) const {
    if (v.size() != other.v.size()) return 0;
    double res = 0;
    for (size_t i = 0; i < v.size(); ++i) res += v[i] * other.v[i];
    return res;
}

CVector CVector::operator*(double scalar) const {
    CVector res(v.size());
    for (size_t i = 0; i < v.size(); ++i) res.v[i] = v[i] * scalar;
    return res;
}

double CVector::module() const {
    double res = 0;
    for (double x : v) res += x * x;
    return sqrt(res);
}

ostream& operator<<(ostream& os, const CVector& cv) {
    os << "(";
    for (size_t i = 0; i < cv.v.size(); ++i) os << cv.v[i] << (i == cv.v.size() - 1 ? "" : ", ");
    return os << ")";
}

istream& operator>>(istream& is, CVector& cv) {
    int n; is >> n;
    cv.v.assign(n, 0);
    for (int i = 0; i < n; ++i) is >> cv.v[i];
    return is;
}

```

- **main.cpp:**

```
#include "CVector.h"
using namespace std;

int main() {
    CVector v1, v2;
    double s;
    if (!(cin >> v1 >> v2 >> s)) return 1;
    cout << "Tong: " << v1 + v2 << endl;
    cout << "Hieu: " << v1 - v2 << endl;
    cout << "Tich vo huong: " << v1 * v2 << endl;
    cout << "Tich giua v1 va hang so: " << v1 * s << endl;
    cout << "Do lon v1: " << v1.module() << endl;
    cout << "Do lon v2: " << v2.module() << endl;
    return 0;
}
```

- **Test cases:**

Test case	Input	Output
1	Vector 1: (1, 0, 9) Vector 2: (5, 0, 4)	<pre>3 1 0 9 3 5 0 4 4 Tong: (6, 0, 13) Hieu: (-4, 0, 5) Tich vo huong: 41 Tich giua v1 va hang so: (4, 0, 36) Do lon v1: 9.05539 Do lon v2: 6.40312 Process returned 0 (0x0) execution time : 5.810 s Press any key to continue.</pre>
2	Vector 1: (1, 2) Vector 2: (3, 4)	<pre>2 1 2 2 3 4 3 Tong: (4, 6) Hieu: (-2, -2) Tich vo huong: 11 Tich giua v1 va hang so: (3, 6) Do lon v1: 2.23607 Do lon v2: 5 Process returned 0 (0x0) execution time : 4.465 s Press any key to continue.</pre>

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

3	Vector 1: (1, 2, 3, 4) Vector 2: (5, 6, 7, 8)	<pre>4 1 2 3 4 4 5 6 7 8 3 Tong: (6, 8, 10, 12) Hieu: (-4, -4, -4, -4) Tich vo huong: 70 Tich giua v1 va hang so: (3, 6, 9, 12) Do lon v1: 5.47723 Do lon v2: 13.1909 Process returned 0 (0x0) execution time : 10.106 s Press any key to continue.</pre>
4	Vector 1: (2, 7, 5) Vector 2: (1, 0, 1, 0, 1)	<pre>3 2 7 5 5 1 0 1 0 1 2.5 Tong: () Hieu: () Tich vo huong: 0 Tich giua v1 va hang so: (5, 17.5, 12.5) Do lon v1: 8.83176 Do lon v2: 1.73205 Process returned 0 (0x0) execution time : 18.299 s Press any key to continue.</pre>

1.7 CMatrix

- Thuộc tính: row, col, data
- Phương thức: phương thức khởi tạo, operator+(), operator-(), operator*(double), operator*(CMatrix), operator*(CVector), transpose(), determinant(), operator<<, operator>>
- Class diagram:

CMatrix

-row: int
 -col: int
 -data: vector<vector<double>>

+CMatrix(r: int, c: int)
 +operator+(other: CMatrix) : : CMatrix
 +operator*(scalar: double) : : CMatrix
 +operator*(other: CMatrix) : : CMatrix
 +operator*(v: CVector) : : CVector
 +transpose() : : CMatrix
 +determinant() : : double
 <> operator<<(os: ostream, m: CMatrix) : : ostream
 <> operator>>(is: istream, m: CMatrix) : : istream

- Code:
- **CMatrix.h:**

```
#ifndef CMATRIX_H
#define CMATRIX_H
#include <iostream>
#include <vector>
#include "CVector.h"

class CMatrix {
private:
    int row, col;
    std::vector<std::vector<double>>> data;
public:
    CMatrix(int r = 0, int c = 0);
    CMatrix operator*(double scalar) const;
    CMatrix transpose() const;
    CMatrix operator+(const CMatrix& other) const;
    CMatrix operator*(const CMatrix& other) const;
    CVector operator*(const CVector& v) const;
    double determinant() const;

    friend std::ostream& operator<<(std::ostream& os, const CMatrix& m);
    friend std::istream& operator>>(std::istream& is, CMatrix& m);
};
#endif
```

- **CMatrix.cpp:**


```

#include "CMatrix.h"
using namespace std;

CMatrix::CMatrix(int r, int c) : row(r), col(c) {
    data.assign(r, vector<double>(c, 0));
}

CVector CMatrix::operator*(const CVector& vec) const {
    if (this->col != (int)vec.v.size()) return CVector(0);

    CVector res(this->row);
    for (int i = 0; i < this->row; ++i) {
        double sum = 0;
        for (int j = 0; j < this->col; ++j) {
            sum += this->data[i][j] * vec.v[j];
        }
        res.v[i] = sum;
    }
    return res;
}

CMatrix CMatrix::operator*(const CMatrix& other) const {
    if (this->col != other.row) return CMatrix(0, 0);

    CMatrix res(this->row, other.col);
    for (int i = 0; i < this->row; ++i) {
        for (int j = 0; j < other.col; ++j) {
            for (int k = 0; k < this->col; ++k) {
                res.data[i][j] += this->data[i][k] * other.data[k][j];
            }
        }
    }
    return res;
}

ostream& operator<<(ostream& os, const CMatrix& m) {
    for (int i = 0; i < m.row; ++i) {
        for (int j = 0; j < m.col; ++j) os << m.data[i][j] << " ";
        os << (i == m.row - 1 ? "" : "\n");
    }
    return os;
}

istream& operator>>(istream& is, CMatrix& m) {
    is >> m.row >> m.col;
    m.data.assign(m.row, vector<double>(m.col, 0));
    for (int i = 0; i < m.row; ++i)
        for (int j = 0; j < m.col; ++j) is >> m.data[i][j];
    return is;
}

```

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

```
    return res;
}

CMatrix CMatrix::operator+(const CMatrix& other) const {
    if (this->row != other.row || this->col != other.col) return CMatrix(0, 0);
    CMatrix res(row, col);
    for (int i = 0; i < row; ++i)
        for (int j = 0; j < col; ++j)
            res.data[i][j] = this->data[i][j] + other.data[i][j];
    return res;
}

CMatrix CMatrix::operator*(double scalar) const {
    CMatrix res(row, col);
    for (int i = 0; i < row; ++i)
        for (int j = 0; j < col; ++j)
            res.data[i][j] = this->data[i][j] * scalar;
    return res;
}

CMatrix CMatrix::transpose() const {
    CMatrix res(col, row);
    for (int i = 0; i < row; ++i)
        for (int j = 0; j < col; ++j)
            res.data[j][i] = this->data[i][j];
    return res;
}

double CMatrix::determinant() const {
    if (row != col) return 0; // Chỉ tính cho ma trận vuông
    int n = row;
    vector<vector<double>> temp = data;
    double det = 1;
    for (int i = 0; i < n; ++i) {
        int pivot = i;
        for (int j = i + 1; j < n; ++j)
            if (abs(temp[j][i]) > abs(temp[pivot][i])) pivot = j;
        swap(temp[i], temp[pivot]);
        if (pivot != i) det *= -1;
        if (abs(temp[i][i]) < 1e-9) return 0;
        det *= temp[i][i];
        for (int j = i + 1; j < n; ++j) {
            double factor = temp[j][i] / temp[i][i];
            for (int k = i + 1; k < n; ++k)
                temp[j][k] -= factor * temp[i][k];
        }
    }
    return det;
}
```

- **main.cpp:**

```

#include "CMatrix.h"
#include "CVector.h"
#include <iostream>
using namespace std;

int main() {
    CMatrix A, B;
    CVector V;

    cout << "Nhap ma tran vuong A (2x2 hoac 3x3): " << endl;
    cin >> A; // Nhap: 2 2 (hang cot) sau do la cac phan tu

    cout << "Nhap ma tran B (cung kich thuoc A): " << endl;
    cin >> B;

    cout << "Nhap vector V (so chieu bang cot A): " << endl;
    cin >> V;

    cout << "\n--- KET QUA ---" << endl;
    cout << "Dinh thuc cua A: " << A.determinant() << endl;
    cout << "A + B:\n" << (A + B) << endl;
    cout << "A * B:\n" << (A * B) << endl;
    cout << "A * V: " << (A * V) << endl;

    return 0;
}

```

- Test cases:

Test case	Input	Output

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

1	Ma trận 1: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ Ma trận 2: $\begin{bmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}$ Vector: (1, 2, 3) Số thực: 4	<pre> 1. Nhap ma tran A (hang cot cac_phan_tu): 3 3 1 2 3 4 5 6 7 8 9 2. Nhap ma tran B (cung kích thước A): 3 3 4 5 6 7 8 9 10 11 12 3. Nhap vector V (số chiều bằng số cột của A): 3 1 2 3 4. Nhap một số thực s: 4 --- KẾT QUẢ KIỂM TRA --- Chuyển vị của A: 1 4 7 2 5 8 3 6 9 Định thức của A: 0 A + B: 5 7 9 11 13 15 17 19 21 A * 4 (Nhân số thực): 4 8 12 16 20 24 28 32 36 A * V (Ma trận nhân Vector): (14, 32, 50) A * B (Ma trận nhân Ma trận): 48 54 60 111 126 141 174 198 222 Process returned 0 (0x0) execution time : 14.425 s Press any key to continue. </pre>
2	Ma trận 1: $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ Ma trận 2: $\begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix}$ Vector: (3, 6) Số thực: 2.5	<pre> 1. Nhap ma tran A (hang cot cac_phan_tu): 2 2 1 0 0 1 2. Nhap ma tran B (cung kích thước A): 2 2 2 0 0 7 3. Nhap vector V (số chiều bằng số cột của A): 2 3 6 4. Nhap một số thực s: 2.5 --- KẾT QUẢ KIỂM TRA --- Chuyển vị của A: 1 0 0 1 Định thức của A: 1 A + B: 3 0 0 8 A * 2.5 (Nhân số thực): 2.5 0 0 2.5 A * V (Ma trận nhân Vector): (3, 6) A * B (Ma trận nhân Ma trận): 2 0 0 7 Process returned 0 (0x0) execution time : 15.274 s Press any key to continue. </pre>

IT002 – LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

3	Ma trận 1: $\begin{bmatrix} 1 & 9 \\ 5 & 4 \end{bmatrix}$ Ma trận 2: $\begin{bmatrix} 0 & 7 \\ 0 & 5 \end{bmatrix}$ Vector: $(7, 2)$ Số thực: 3	<pre> 1. Nhap ma tran A (hang cot cac_phan_tu): 2 2 1 9 5 4 2. Nhap ma tran B (cung kich thuoc A): 2 2 0 7 0 5 3. Nhap vector V (so chieu bang so cot cua A): 2 7 2 4. Nhap mot so thuc s: 3 --- KET QUA KIEM TRA --- Chuyen vi cua A: 1 5 9 4 Dinh thuc cua A: -41 A + B: 1 16 5 9 A * 3 (Nhan so thuc): 3 27 15 12 A * V (Ma tran nhan Vector): (25, 43) A * B (Ma tran nhan Ma tran): 0 52 0 55 Process returned 0 (0x0) execution time : 26.927 s Press any key to continue. </pre>
---	---	---

Link GitHub: <https://github.com/vexigamaker/OOPLab4>

(do em mới thay đổi điện thoại, điện thoại mới chưa cài MS Authenticator và tài khoản GitHub bằng Gmail trường bật 2FA bằng MS Authenticator nên chưa thể đăng nhập được, em xin phép đăng lên GitHub cá nhân)